



Relatório Correção Bubble-Sort

Carlos Eduardo Souza - 211061510

BRASÍLIA - DF, 08 de Julho de 2026

1. Introdução

Este relatório documenta o desenvolvimento da formalização da correção do algoritmo Bubble Sort no assistente de provas Coq (versão 8.20), conforme proposto no trabalho da disciplina. O projeto consistiu em definir a função `bubble` (uma passagem de borbulhamento), a função `bs` (o algoritmo completo), e provar formalmente que `bs` produz uma lista ordenada que é permutação da entrada (teorema `bs_correto`).

2. Erros identificados na proposta original

O enunciado do trabalho propunha originalmente o seguinte lema sobre `bubble`:

Lemma `bubble_sorted`: $\forall l, \text{Sorted } l \rightarrow \text{bubble } l = l$.

Esse lema afirma que aplicar `bubble` a uma lista já ordenada não a altera (é a identidade). Esse lema está correto e foi provado sem problemas, ele de fato decorre diretamente da definição de `bubble` via `functional induction`.

O erro real estava em uma suposição implícita do enunciado, sugerida pela descrição textual do algoritmo: a ideia de que `bubble`, aplicado a uma lista qualquer, levaria o menor elemento global para a cabeça do resultado (uma leitura plausível de "borbulhamento", por analogia com o nome do algoritmo). Essa suposição é falsa, e isso foi confirmado computacionalmente já no próprio enunciado:

Eval compute in `bubble (3::2::1::nil)`.

= `2 :: 1 :: 3 :: nil`

Note que o resultado não tem 1 (o mínimo) na cabeça — tem 2. Isso ocorre porque `bubble` compara apenas pares consecutivos: ao comparar 3 e 2, como `3 > 2`, o 3 é empurrado para a posição seguinte e carregado adiante nas comparações futuras; o 2, por sua vez, nunca é comparado novamente com o 1, pois já ocupou a cabeça da lista. Assim, `bubble` garante apenas uma propriedade local (empurrar o maior de cada par adjacente para frente), e não uma propriedade global sobre o mínimo da lista.

Esse erro de leitura tinha uma consequência direta sobre a estratégia de prova: uma primeira tentativa de demonstrar um lema do tipo `bubble_hd_min` (afirmando que a cabeça de `bubble l` é o mínimo de `l`) falhou e tinha que falhar, pois o enunciado é simplesmente falso, como o próprio exemplo computacional

acima demonstra. A identificação desse erro foi o ponto de virada do projeto: a partir daí, ficou claro que era necessário enunciar um invariante mais cuidadoso, local e relativo à cauda, em vez de uma propriedade global sobre a cabeça.

3. Adaptações realizadas no código

Em decorrência do erro discutido acima, a estratégia de prova original (baseada num lema de "mínimo na cabeça") foi abandonada e substituída pela seguinte adaptação, que constitui a principal contribuição metodológica deste projeto:

3.1 O invariante correto: `bubble_preserves_sorted`

Em vez de provar algo sobre a cabeça de `bubble l`, formulamos e provamos o seguinte lema, mais geral e mais fraco em aparência, mas suficiente para o objetivo final:

Lemma `bubble_preserves_sorted`: $\forall l, \text{Sorted } l \rightarrow \forall c, \text{Sorted } l \rightarrow \text{Sorted } (\text{bubble } (c :: l))$.

Ou seja: se a cauda `l` já está ordenada, então `bubble (c :: l)` também está ordenada, para qualquer elemento `c` adicionado à frente sem exigir que `c` termine na cabeça do resultado. Essa é exatamente a propriedade que `bs` precisa: em `bs (h::l') = bubble (h::(bs l'))`, sabemos por hipótese de indução que `bs l'` está ordenada, mas nada garante a relação entre `h` e os elementos de `bs l'`. O lema `bubble_preserves_sorted` resolve precisamente essa lacuna, tornando a prova de `bs_sorted` quase imediata.

A prova deste lema foi feita por indução estrutural em `l` (e não por `functional induction` sobre `bubble`), pois é necessário generalizar sobre `c` a cada passo recursivo, algo que `functional induction` não oferece diretamente, já que a recursão de `bubble` alterna o papel de `c` e da cabeça de `l` de uma chamada para a outra.

3.2 Dois lemas auxiliares introduzidos

Para viabilizar a prova de `bubble_preserves_sorted`, foram introduzidos dois lemas auxiliares que não constavam do enunciado original:

- `bubble_HdRel_le`: se um elemento `a` é $\leq$ a todos os elementos de uma lista `l`, então `a` também é $\leq$ a todos os elementos de `bubble l`, em particular, à sua cabeça. A prova explora o fato de que `bubble` apenas

permuta os elementos (via `bubble_perm` e `Permutation_in`), logo qualquer cota inferior válida para `l` continua válida para `bubble l`.

- `forall_le_a_l`: se `a :: l` está ordenada (`Sorted le`), então `a` é $\leq$ a todos os elementos de `l`, não apenas ao elemento imediatamente seguinte. Esse fato não decorre diretamente de `Sorted`, que só garante a relação entre elementos adjacentes; foi necessário usar a conversão `Sorted_StronglySorted` (fornecendo transitividade de `le`, resolvida por `lia`) para obter a versão "forte" da ordenação, da qual a propriedade desejada sai por inversão.

Esses dois lemas, combinados, permitem mostrar, nos dois ramos do `if` dentro de `bubble_preserves_sorted`, que o elemento que acaba na cabeça do resultado de `bubble (c :: a :: l '' )` é sempre $\leq$ a todos os elementos restantes, por transitividade, mesmo sem saber a priori qual dos dois elementos (`c` ou `a`) é esse.

3.3 Ajustes técnicos pontuais

Além da adaptação estrutural acima, algumas correções técnicas menores foram necessárias durante o desenvolvimento, todas decorrentes de particularidades da versão do Coq utilizada (8.20):

- A tática `functional induction (bubble l) as [...]` (com padrões nomeados) não está disponível nessa versão; foi necessário usar `functional induction (bubble l) sem as`, deixando os nomes serem atribuídos automaticamente.
- Para manipular as hipóteses booleanas geradas pelo `if x <=? y` (extraídas automaticamente por `functional induction` como hipóteses do tipo `x <=? y = true/= false`), as lemas corretos da biblioteca padrão são `leb_complete` e `leb_complete_conv` (e não `Nat.leb_le`, que se aplica a um tipo de hipótese ligeiramente diferente).
- Na prova de `bubble_sorted`, foi necessário usar `HdRel_cons/inversão` explícita em vez do uso direto do construtor `constructor` em alguns pontos, devido à forma como `Sorted` é definido indutivamente.

4. Conclusão

O teorema final `bs_correto` foi demonstrado com sucesso, estabelecendo que, para toda lista `l`, `bs l` é simultaneamente ordenada (`Sorted le (bs l)`) e uma permutação de `l` (`Permutation l (bs l)`). A verificação `Print Assumptions bs_correto` confirma que a prova está "fechada sob o contexto

global" (Closed under the global context), isto é, não depende de nenhum axioma adicional além do núcleo lógico do Coq.

O principal aprendizado do projeto foi metodológico: a tentativa inicial de seguir literalmente a propriedade sugerida pelo enunciado (mínimo na cabeça) levou a um beco sem saída, e só a verificação computacional direta (Eval compute) revelou que essa propriedade não vale. A solução exigiu reformular o invariante central em termos mais fracos, mas corretos e suficientes, uma lição importante sobre a diferença entre a intuição informal de um algoritmo e a propriedade formal que ele de fato satisfaz.